

Week 5: Longest Increasing Subsequence

Questions

1. Design an algorithm to find $h(i)$. Can your algorithm work in $O(\log_2 i)$ time?

使用二分搜索，在排序的數列中找到最後一個小於等於 a_i 的數字的索引。
時間複雜度為 $O(\log_2 i)$ 。

2. Design an algorithm to find $f(i)$.

$f(1) = 1, g(1) = 1, h(1) = 0$
若 $g(i) > f(i-1)$, $f(i) = g(i)$
否則 $f(i) = f(i-1)$ 。
時間複雜度為 $O(n)$

3. Design an algorithm to reconstruct the subsequence that length is $f(i)$.

尋找最大值 $h(j), i \geq j$, 子序列為 $B = b_1, b_2 \dots b_{g(j)}$
 $b_{g(j)} = a_{h(j)}, b_{n-1} = a_{h(n)}$

4. Analyze space complexity and time complexity of algorithms in 1~3.
5. Solve ALG04B on <http://oj.csie.ndhu.edu.tw/>

C++

```
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    int n;

    while (cin >> n)
    {
        vector<int> str(n);
        for (int i = 0; i < n; ++i)
            cin >> str[i];

        vector<int> lis;
        vector<int> lds;
        lis.emplace_back(str[0]);
        lds.emplace_back(str[n - 1]);

        vector<int> num1(n);
        vector<int> num2(n);
        num1[0] = 1;
        num2[n - 1] = 1;
```

```
int cnt1 = 1, cnt2 = 1;

for (int i = 1; i < n; ++i)
{
    //lis
    if (str[i] > lis.back())
    {
        lis.emplace_back(str[i]);
        num1[i] = ++cnt1;
    }
    else
    {
        auto it = lower_bound(lis.begin(), lis.end(), str[i]);
        *it = str[i];
        num1[i] = (it - lis.begin() + 1);
    }

    //lds
    int j = n - i - 1;
    if (str[j] > lds.back())
    {
        lds.emplace_back(str[j]);
        num2[j] = ++cnt2;
    }
    else
    {
        auto it = lower_bound(lds.begin(), lds.end(), str[j]);
        *it = str[j];
        num2[j] = (it - lds.begin() + 1);
    }
}

int t = min(cnt1, cnt2);

for (; t > 0; --t) for (int i = 0; i < n; ++i)
    if (num1[i] >= t && num2[i] >= t) goto out;

out:
    cout << t * 2 - 1 << "\n";
}
```